TRINITY: Exploiting Latency Sensitivity to Improve Quality of Experience on Cloud VR Gaming

Lingzhi Zhao* Yongqiang Gui† Yanyan Suo†
Sandesh Dhawaskar Sathyanarayana† Ruixiao Zhang† Klara Nahrstedt* Shu Shi†

*University of Illinois Urbana-Champaign, †Bytedance

Abstract

In this paper, we present TRINITY, a novel and practical solution that enables dynamic game streaming from cloud servers to virtual reality (VR) headsets. Achieving the stringent motion-to-photon (MTP) latency requirement of under 25ms presents a critical challenge in VR game streaming. Existing approaches primarily support specific actions (e.g., head movement), resulting in an *MTP latency gap* with other actions subject to round-trip delays, thereby degrading overall user quality of experience (QoE). To bridge the MTP latency gap, TRINITY proposes a novel approach grounded in the observation that *player interactions exhibit heterogeneous latency requirements*. Specifically, we classify user inputs into three distinct motion categories: *head motion*, *hand motion*, and *body motion*. By aligning latency levels with users' perceptual sensitivity to different types of motion and strategically introducing controlled delays to the hand motion, TRINITY mitigates the perceptual impact of latency discrepancy across motions. We evaluate TRINITY with a highly latency-sensitive first-person shooting game streamed from a remote server to a VR headset. Real-world user studies demonstrate that TRINITY delivers a user experience comparable to a local streaming setup, despite operating under network-induced delays.

CCS Concepts

• **Human-centered computing** → **Virtual reality**; • **Software and its engineering** → **Interactive games**.

Keywords

Virtual reality, cloud VR gaming, MTP latency

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
MMSys '26, April 4–8, 2026, Hong Kong, Hong Kong
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2481-7/26/04…$15.00
https://doi.org/10.1145/3793853.3795744

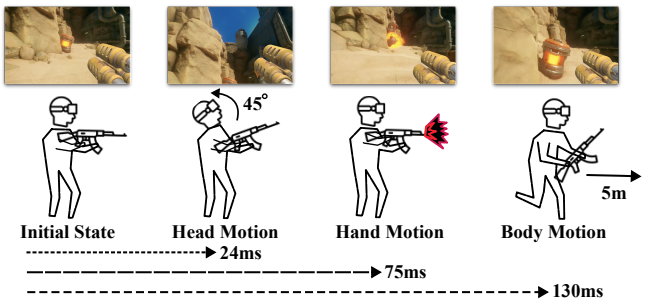


Figure 1: The idea of TRINITY: delaying hand motion to bridge the MTP latency gap between head and body motions.

1 Introduction

The emergence of virtual reality (VR) headsets has revolutionized user experiences, driving rapid growth in the entertainment industry, particularly in VR gaming [17, 43]. Recent products [18, 39] allow game content to be streamed wirelessly from powerful GPUs, elevating the gaming experience to new heights. However, such VR streaming is largely confined to the same LAN due to stringent latency requirements. Specifically, motion-to-photon (MTP) latency—the delay between a user's action and the corresponding visual update—must remain below 25 ms to ensure a satisfactory gaming experience [25, 30, 44]. Directly offloading rendering to a remote cloud server introduces excessive latency, severely degrading MTP performance and diminishing users' quality of experience (QoE).

To address these challenges, prior work has proposed several latency-mitigation techniques for VR streaming systems. General-purpose approaches [10, 29] aim to reduce latency across the entire video transmission and processing pipeline, thereby lowering overall MTP. More specialized solutions target specific interaction types. For 3-DOF (degrees of freedom) actions, which involve only changes in viewing direction, servers can generate 360° panoramas of each scene, enabling headsets to render local views in real time [27, 42]. For 6-DOF actions, which also include positional changes, servers must provide additional auxiliary data (e.g., depth maps) so that headsets can perform DIBR (depth image-based rendering) to synthesize frames locally [35, 50]. For other user interactions such as weapon firing, researchers in [28] proposed pre-rendering probable future actions on the server and streaming all possible outcomes to the client, allowing immediate display when the predicted action occurs.

Despite these advances, many of these techniques are impractical for VR game streaming, where scenes must be rendered dynamically at runtime rather than pre-processed. As we elaborate in §2.2, neither computing nor networking resources suffice to pre-render all possible actions on the server. Consequently, although certain latencies can be reduced by enabling local rendering on the headset, a persistent *MTP latency gap* remains—a discrepancy between the near-instantaneous response to head movement and the delayed feedback from server-rendered interactions. Under prolonged network delays, this gap widens and degrades the user experience [30, 42].

In this paper, we propose Trinity, a novel and practical solution to bridge the MTP latency gap to enhance cloud VR gaming experiences. Our approach is motivated by the observation that users exhibit varying latency tolerance for different actions. This heterogeneity in QoE sensitivity suggests that applying a single stringent MTP latency requirement to all motions is unnecessary. Instead, Trinity classifies user actions into three categories: head, body, and hand motions (illustrated in Fig. 1), each with differing perceptual sensitivities to latency. Head motion, being most latency-sensitive, is rendered locally on the VR headset to maintain the critical sub-25ms MTP latency [9, 22]. Body motion, which encompasses avatar movements and other game actions requiring server computation, inherently involves full network round-trip delays. The key innovation is in hand motion: by locally rendering foreground elements (like held objects) while introducing controlled delays, we strategically align its MTP latency between head and body motions, thereby minimizing perceptual discrepancies and preserving QoE.

To validate the effectiveness of Trinity, we implemented a complete streaming system and deployed it in an open-source first-person shooting (FPS) game [16], which is a genre widely recognized as the most latency-critical [23] and popular [48]. Our extensive subjective study with real users demonstrated that Trinity successfully achieves its design goals. Even with an average network delay of 130ms between the cloud rendering server and VR headset, Trinity achieved an average mean opinion score (MOS) of 4.0 for the overall gaming experience, even slightly outperforming the 3.9 MOS rating obtained by ALVR [11] (the state-of-the-art VR streaming solution) in a wireless local area network (WLAN) setup. These results confirm Trinity's potential for broader VR streaming applications.

To summarize, our key contributions include:

- We validate a fundamental insight that users have varying latency tolerance for different types of actions, and develop a novel solution that successfully bridges the MTP latency gap in cloud VR gaming systems.
- We design and implement Trinity as an end-to-end cloud VR gaming system supporting latency-sensitive FPS games.
- Through extensive user studies, we show that the gaming experience of Trinity is comparable, if not better than a local streaming version.

2 Background and Motivation

2.1 Cloud VR Gaming

Cloud VR gaming refers to a streaming system where game content is rendered on cloud servers and transmitted as video frames to VR headsets. This paper specifically addresses the challenges of streaming dynamic games that require continuous rendering, as opposed to static games that permit preprocessing optimizations.

Overview of VR streaming. A typical VR streaming system comprises two essential components: a server executing the VR application, and a VR headset serving dual input-output functions. The VR headset utilizes integrated sensors (i.e., gyroscopes, accelerometers, etc.) and bluetooth controllers [20, 36, 37] to capture user actions including head orientation, hand gestures, and so on. These inputs are transmitted to the server where they undergo processing through the VR application pipeline, subsequently generating corresponding rendered frames that are encoded and streamed back to the client-side VR headset. There have been several streaming solutions including ALVR [11], Steam Link [6], and proprietary equivalents from different VR providers, though we note that most solutions typically restrict operation to WLAN setups only.

Static *vs.* dynamic. We classify VR games into two categories depending on whether the game scene is static or dynamic. Static environments maintain visual consistency except when modified by direct user interaction, such as position changes during navigation. This stability enables additional optimization opportunities, including offline pre-rendering of scene variations [13] or machine learning-based sequence prediction for runtime efficiency [50]. In contrast, dynamic environments exhibit continuous transformation independent of user input, requiring real-time server rendering of each frame regardless of user activity. The streaming complexity between these categories differs substantially. Dynamic scenes demand persistent computational resources for continuous frame generation and transmission, making it impractical to apply static-scene optimization methods. *Our research focuses on dynamic scene streaming, which presents more challenging problems in cloud VR systems.*

Video frame rate *vs.* display refresh rate. Critical to understanding VR streaming is the distinction between two fundamental rate parameters: the video frame rate and the display refresh rate. The video frame rate represents the frequency at which the server renders and transmits game frames to the headset, constrained by both GPU rendering capability and network bandwidth availability.

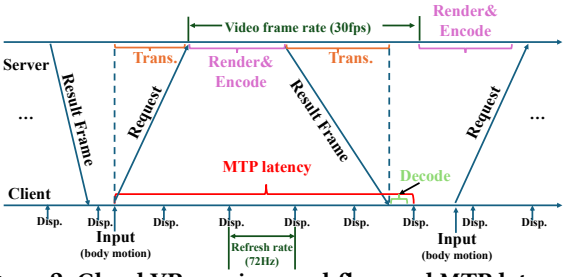
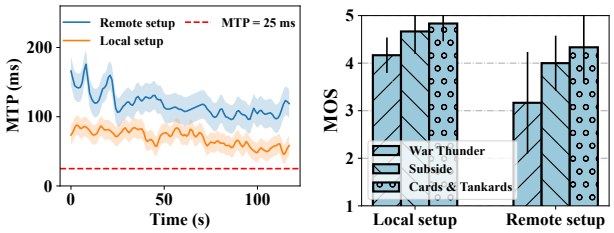


Figure 2: Cloud VR gaming workflow and MTP latency illustration of remote rendering.



(a) MTP latency. **(b) Game experience.**

Figure 3: Cloud deployment exhibits highly variable MTP latencies and leads to low QoE.

Conversely, the display refresh rate denotes how frequently the display hardware of VR headset updates the screen, determined by the headset's local processing capacity and display technology specifications. Contemporary headsets typically maintain a minimum refresh rate of 72Hz [39]. Since the display subsystem discards any frames exceeding its refresh capacity, it is usually not necessary to configure video frame rate beyond the display's refresh capability to avoid wasting computational and network resources.

MTP latency. MTP latency represents a critical performance metric in VR systems, formally defined as the temporal interval between a user input's reception by the headset and the visual presentation of its corresponding rendered frame. Extensive research [14, 25] has established 25ms as the maximum acceptable threshold for maintaining user comfort and presence in VR environments.

In the scenario where the visual feedback can be rendered locally on the VR headset, achieving this threshold requires careful synchronization between input processing and display subsystems. A 72Hz [39] display refresh rate (corresponding to 13.8ms per frame) produces an effective MTP latency range of 13.8-27.6ms, depending on the relative timing between input events and display refresh cycles. This yields an average latency of 21ms, marginally matching the established comfort threshold. However, for the user actions that can only be rendered on the server, the MTP latency encompasses additional components, including the network propagation delay, server-side rendering time, video encoding and decoding time, and the post-processing time on the VR headset. Fig. 2 shows an illustration of these MTP latencies in the remote rendering setup. These additive factors substantially increase total MTP latency compared to the local rendering setup, creating significant challenges for maintaining the sub-25ms threshold in cloud VR deployments.

Cloud VR experience. Existing VR streaming solutions prove inadequate for cloud deployment due to fundamental architectural constraints. When all rendering is performed on remote servers, the resulting MTP latency becomes dominated by network propagation delays, leading to inconsistent performance that severely degrades user experience. To quantify these effects, we conducted a comparative study evaluating two deployment scenarios: (1) a local configuration with co-located server and VR headset within a WLAN, and (2) a cloud configuration with the server hosted in a remote datacenter. Both implementations utilized ALVR [11], an open-source VR streaming solution, to assess three representative game genres: the first-person shooting game War Thunder [3], the diving simulator Subside [1], and the strategy game Cards & Tankards [2]. The games are selected to cover a range of VR content with diverse spatial and temporal complexity [47]. For example, FPS games are among the most dynamic applications that involve frequent interactions and scene updates, whereas card games are comparatively static with slower visual changes. Participant evaluations followed the experimental protocol detailed in §5.

Our results in Fig. 3a demonstrate latency disparities between the two scenarios. The cloud deployment exhibited highly variable MTP latencies (91-238ms) that substantially impaired user experience across all tested games as shown in Fig. 3b. While the local WLAN configuration showed more consistent performance, its average 80ms latency still exceeded the 25ms threshold required for optimal VR experience. This was particularly evident in the FPS game, where latency sensitivity aligns with established perceptual research [30]. These findings motivate us to focus investigation of FPS game optimization in cloud VR environments, as this genre presents both the most stringent latency requirements and the greatest potential for QoE improvements through our proposed techniques.

2.2 Limitations of Existing Solutions

Several techniques [7, 8, 10, 21, 24, 29, 32, 40, 49, 51] have been proposed to address the MTP latency challenge in cloud gaming and VR streaming systems. These approaches can be broadly classified into the following three categories.

Frame synthesis at client side. The first approach leverages the VR headset's rendering capability to process certain user actions locally, which can reduce its average MTP latency to less than 25ms when combined with a 72Hz display refresh rate [13, 27, 28, 31, 33, 35, 41, 42, 50]. For example, Furion [27] streams panoramic image frames, enabling the VR headset to locally synthesize views for head movements

(3-DOF) using the provided panorama. Similar work has been done in Freedom [42] and MEC-VR [41]. Other systems like ZGaming [50] utilize DIBR techniques to generate new views that accommodate positional changes in the virtual environment (6-DOF).

Pre-rendering at server side. The second approach involves server-side prediction of user actions with pre-rendering of anticipated game scenes [27, 28, 50, 51]. When predictions are accurate, the headset can display pre-fetched content with minimal additional latency. Outatime [28] anticipates all possible user-triggered game states within a 256 ms window and delivers the rendered frames for each possibility to the client. ZGaming [50] employs a neural network to predict future movements, while Furion [27] pre-generates multiple panoramic frames that correspond to adjacent user positions, allowing the client to select and display the most appropriate frame when its position changes.

Latency reduction. Researchers have also explored approaches that directly reduce end-to-end latency in VR streaming [7, 8, 10, 15, 21, 24, 29, 32, 40, 49]. For example, MoVR [7, 8, 40] leverages millimeter-wave technology to achieve high-throughput VR delivery, while Cutting the Cord [29] proposes a parallel rendering and streaming pipeline to minimize latency. In the meantime, researchers [10, 21, 32] also focused on the optimization of projection and encoding to reduce the transmission latency. For example, Foveated coding or spherical projection [49] strategically allocates bits or pixels to perceptually important visual areas, respectively, to reduce the transmission data size.

Limitations. Trinity aims to mitigate the perceived impact of latency on user experience rather than directly reducing end-to-end delay. Therefore, we primarily compare Trinity against the first two categories of prior work that focus on latency masking. As for the third category, these techniques can be applied on top of Trinity to further reduce latency. They are orthogonal to our design and can be readily integrated into Trinity. Despite the benefits of existing latency-masking techniques, they have significant limitations when applied to cloud VR gaming systems. First, for games with dynamic scenes requiring continuous rendering at 60fps, generating and streaming additional data for local rendering can be computationally expensive yet still feasible. This is because, at each render cycle, the server must produce a panoramic or depth-based representation of the game scene. As shown in Fig. 4, creating and streaming a cubemap panorama demands 2.5x more computational resources and 4.0x more network bandwidth compared to transmitting only the user's viewport. Second, pre-fetching pre-rendered frames for speculative future actions proves impractical in game streaming. The dynamic nature of game environments, particularly in multiplayer settings, makes it impossible to predict all possible outcomes accurately. Moreover, the server does not have enough computing resources to perform speculative rendering, especially when it is already exhausted to generate supplement data for local rendering on VR headsets.

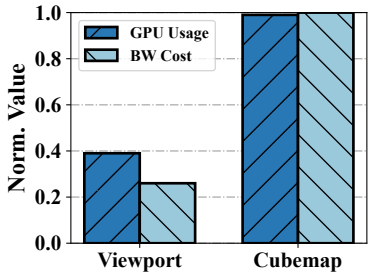


Figure 4: Higher GPU and bandwidth costs for rendering and transmitting a cubemap.

Therefore, the most viable optimization strategy for cloud VR gaming involves leveraging server-side computational resources to generate supplemental data that enables local rendering on the VR headset. While this approach can effectively reduce MTP latency for supported actions to the required threshold, it creates a latency gap, as other unsupported actions suffer round-trip network delay.

2.3 MTP Latency Gap

We conducted a user study to evaluate how this MTP latency gap affects user experience. Using an open-source FPSSample game [16], we modified the game logic to support only three user actions (head moving, gun aiming, and weapon firing) and introduced controlled delays to simulate different latency conditions for these actions. The game ran on a tethered VR headset to eliminate network interference, allowing precise measurement of latency effects. We invited actual users[1] to help analyze how user's MOS score changes as we increase the MTP latency.

Fig. 5 presents our findings. The dash line (Case 1) represents conventional VR streaming where all actions are rendered on the server and experience identical MTP latency. As expected, the MOS score drops sharply from 5 (at 24ms) as the MTP latency increases to 104ms. The dot line (Case 2) shows the optimized solution with local rendering for head movements (fixed at 24ms) while gun actions require server rendering and experience increasing MTP latency (up to 104ms). While this approach maintains higher MOS scores than Case 1 across all tested conditions, the MOS scores still degrade below 3 when the MTP latency gap reaches 80ms (104ms for gun actions versus 24ms for head movements). These results demonstrate that while local rendering for head movements improves overall experience, the growing discrepancy of MTP latencies between it and gun actions ultimately deteriorates QoE.

[1]All user studies presented throughout this paper adhere to the procedure and protocol described in §5.1.

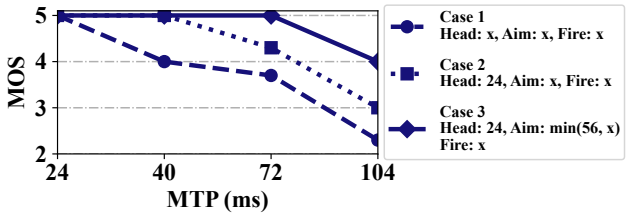


Figure 5: MOS *vs.* MTP latency. "Head"", "Aim"", and "Fire" represent head movement, gun aiming, and weapon firing, respectively. Their MTP latencies are set either as x-axis value (x), 24ms, or min{x,56ms}.

This naturally raises the question: can we further enhance the gaming experience? The solid line (Case 3) in Fig. 5 provides promising insights. Building upon Case 2's optimizations, this approach enables local rendering for gun-aiming action on the VR headset while reserving only weapon-firing action for server rendering. Crucially, rather than displaying immediate visual feedback for gun movements, the system intentionally introduces a controlled 56ms latency for aiming actions when firing latency is higher. This creates a three-tier MTP latency structure: in the maximum delay test condition, the MTP latencies measure 24ms (head movement), 56ms (gun aiming), and 104ms (weapon firing) respectively. Remarkably, this configuration demonstrates superior performance, with the blue line substantially outperforming the other cases. Most notably, even under maximum latency conditions, the MOS score maintains a rating of 4, indicating a consistently acceptable gameplay experience for users.

3 TRINITY Design

3.1 Bridge the Latency Gap

Our findings in §2.3 are not coincidental, but rather stem from fundamental characteristics of human latency perception and tolerance. The human brain is particularly sensitive to head movements because they directly alter the visual input to our eyes. The 25ms MTP latency requirement for VR headsets closely matches the 26-30ms latency of neural visual signal transmission [12]. This aligns with prior physiological research [9, 22] showing that head movements are tightly coupled with the vestibular system, making them especially vulnerable to latency-induced motion and orientation perception disruptions. However, humans show greater tolerance for latency in other types of actions that take longer time to drive body muscles. The average human reaction time for physical movements ranges between 200-250ms [5], which helps explain why cloud gaming users can achieve satisfactory experiences with MTP latency up to 100-150ms. Furthermore, hand movements typically have shorter latency thresholds than full-body movements, as they involve fewer muscle groups and simpler coordination requirements [26].

Building upon these insights, we propose TRINITY, an innovative cloud VR gaming solution that responds to different user motions with appropriate latency levels matching their perceptual tolerance. For the most sensitive *head motions*, TRINITY ensures MTP latency remains below 25ms by performing local panoramic image rendering directly on the VR headset. Most other in-game actions involving body movement or controller operation are classified as *body motions*, which are less latency-sensitive. These actions are rendered server-side and experience full round-trip network delay. Note that TRINITY does not send depth images to the VR headset like ZGaming [50] because the position movements are considered body motion and do not need to render locally.

What distinguishes TRINITY from conventional approaches is its introduction of *hand motions*: actions involving only hand movements whose visual feedback can be rendered locally on the VR headset. These motions typically trigger foreground object animations without altering background scenes. For instance, when a player pulls the controller trigger to fire a weapon, the immediate gun recoil animation qualifies as hand motion, while the subsequent environmental explosion (requiring server rendering) constitutes body motion. Additional examples of these motion classifications in TRINITY are detailed in Table 1.

In TRINITY, we strategically utilize hand motions to bridge the MTP latency gap between immediate head motions and delayed body motions. While hand motions could be rendered instantly on the VR headset (like head motions), we intentionally introduce a controlled delay in their rendering. This ensures the hand motion latency falls between the fast head motions (≤25ms) and slower body motions (full network delay), effectively smoothing the transition of game actions and mitigating the perceptual impact of high body motion latency on QoE. Consider the gunfire example when the player presses the trigger. While the environmental explosion (body motion) takes 130ms to appear, displaying the gun recoil animation (hand motion) at 70ms creates the perception that the entire system responds within 70ms. This temporal alignment helps maintain responsiveness despite the unavoidable server delays for environmental updates.

3.2 Hand Motion Latency Adaptation

While the delayed hand motion can potentially improve players' QoE under high round-trip network latency, how much delay should be added remains an unanswered question. We use an intuitive yet effective linear adaptation strategy as follows:

$$MTP_{hand} = \begin{cases} MTP_{head} & \text{if } MTP_{body} < MTP_{sat}, \\ \alpha \cdot MTP_{head} + \beta \cdot MTP_{body} & \text{if } MTP_{sat} \le MTP_{body} < MTP_{unsat}, \\ MTP_{body} & \text{if } MTP_{body} \ge MTP_{unsat}, \end{cases} \tag{1}$$

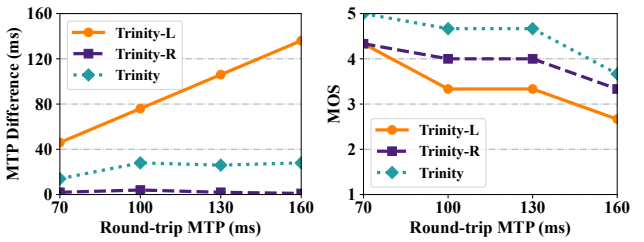
where α and β are non-negative weights associated with head MTP and body MTP, respectively, and satisfy $\alpha + \beta = 1$. A relatively small α indicates that we favor adding more MTP

Actions of Avatars	Actions of Players	Updates of Game Scene	Motion Types
(a) Move avatar's head to look around	Move head	Changes of background scene; on-hand weapon moves accordingly to the view changes	Head motion, hand motion
(b) Move weapon to aim the target	Move (right) joystick	Position changes of the weapon	Hand motion
(c) Fire the weapon	Push (right) trigger button	(i) Discharge and recoil of a weapon; (ii) updates of the background scene that show the fire effect	(i) Hand motion; (ii) body motion
(d) Throw a grenade	Push (left) trigger button	(i) Swing the arm forward and release the grenade; (ii) grenade flies and explodes in the distance	(i) Hand motion; (ii) body motion
(e) Control avatar to move around	Move (left) joystick	Changes of background scenes	Body motion

Table 1: Examples of players' motions in Trinity.

for hand motion to align with body motion. MTP_{sat} represents the maximum round-trip network delay that players still experience satisfactory game experience. Therefore, it is unnecessary to add the delays for hand motion in this case. In contrast, MTP_{unsat} represents the minimum round-trip network delay that exceeds participants' tolerance levels and adding any latency to hand motion does not preserve QoE.

The key idea of the adaptation strategy is to dynamically align the hand motion MTP between head and body motions so that it can always bridge the MTP discrepancy in the presence of varying network conditions. Fig. 6 illustrates the intuition. Specifically, we use two baselines, i.e., Trinity-L and Trinity-R. Trinity-L renders hand motion without introducing additional delay, i.e., aligning it directly with head motion, and Trinity-R synchronizes hand motion latency with body motion latency. Fig.6a and Fig. 6b report the MTP differences between hand motion and round-trip network delay (i.e., body MTP) and the players' QoE, respectively. As shown in Fig. 6a, Trinity introduces a certain delay to hand motion and dynamically adjusts it based on network latency to maintain a relatively consistent latency difference between hand motion and body motion. This improves the smoothness of the overall playback experience, and thus Trinity can achieve higher MOS scores indicated in Fig. 6b. Furthermore, we surprisingly find that Trinity-R performs better than Trinity-L. Since hand motion (e.g., firing a gun) and body motion (explosion effect in the background environment) are highly related, the results highlight that reducing the MTP discrepancy among motions is more effective than the MTP latency for a certain motion. It is important to note that the formulation of the equation and its parameters can be adapted to user preferences and game types. In §5, we detail how the parameters are derived via empirical experiments.



(a) MTP difference. **(b) Game experience.**

Figure 6: Comparisons of latency adaptation strategies. Trinity-L keeps hand motion MTP within 24ms. Trinity-R sets hand motion MTP the same as one for body motion (i.e., round-trip delay). (a) MTP difference between hand motion and body motion. (b) Players' QoE versus round-trip MTP.

3.3 Trinity Architecture

Fig. 7 shows the high-level architecture of Trinity, which consists of a client and a cloud server. The client uses an untethered VR device and connects to the server over a wireless network. In a typical interaction, the client sends control signals to the server (e.g., for movement or shooting). The server then processes these inputs by rendering, encoding, and streaming the corresponding frame back to the client. Upon receiving all the frame packets, the client decodes and displays the frame on the VR headset. These conventional modules are shown in light blue and yellow, while the orange components highlight Trinity's novel features. The design rationale and functionality of these components are detailed in the following section.

First, the client performs motion categorization (§3.1) based on the users' actions where the user inputs are classified into three motion categories individually: head, hand, or body motions. The key innovation of Trinity is the introduction of hand motion (§3.1 & §3.2) where the foreground objects for the hand motion are locally updated and rendered with a controlled delay. In the case of head motion, the client locally synthesizes the new viewport frame based on the most recently received partial panorama [42] from the cloud. For body motion, each new frame is generated and sent from the cloud, and decoded by the client. The background scene frames corresponding to head and body motions are integrated with the foreground objects. The final frame is displayed on the VR headset. On the cloud server side, Trinity renders a partial panorama frame as in [42]. Each rendered frame is encoded and sent as media packets to the client.

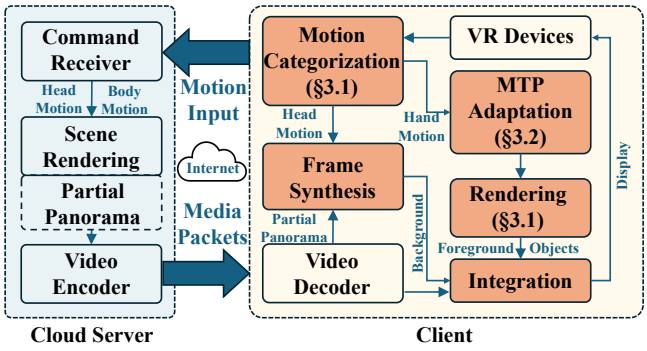


Figure 7: TRINITY overview.

4 Implementation

We implemented TRINITY on top of an open-source multiplayer shooting game, FPSSample [16], which is developed by Unity on the windows platform. FPSSample has rich scene resources and well-established interaction logic. We leveraged FPSSample to develop TRINITY system by separating its functionalities into server and client components.

4.1 Client

Latency measurement. The client converts player inputs into an integer (*motion index*) and sends it to the cloud server to track request lifecycles and measure round-trip network delay. Upon receiving the encoded bitstream of the corresponding frame from the server, the client forwards it to the decoder for processing. As our client device (PICO [39]) runs an Android-based system, we use MediaCodec to decode the H.265 video stream. Metadata such as motion indices and timestamps are parsed for hand motion latency adaptation.

Client rendering. Once decoded, video frames are projected onto a skybox using a custom shader. These frames contain exclusively server-rendered background content corresponding to head and body motions. TRINITY extracts foreground objects like guns and crosshairs related to hand motion and renders them locally on the client device. To realize the delayed hand motion update, transform data, i.e., the position and rotation, of the foreground objects are stored in a queue. Once hand motion latency is determined based on equation 1, the system composites the foreground using historical transforms from the buffer, reprojecting them onto the current background frame. Since the server does not render the gun and crosshair anymore, the foreground objects can be seamlessly integrated with the background scene. We implemented them in Unity and built a customized Android APK that can be directly installed and run in VR headset.

4.2 Server

VR interactions. To extend the FPSSample for VR, we map traditional keyboard and mouse inputs to VR controller interactions, covering head rotation, body movement, and hand actions (e.g., firing). Headset rotation parameters (pitch, yaw, roll) directly control camera orientation. Body movement commands from the VR controller are translated into world coordinates to update the camera position. Finally, shooting actions originally triggered by keyboard inputs are replaced by VR controller button presses.

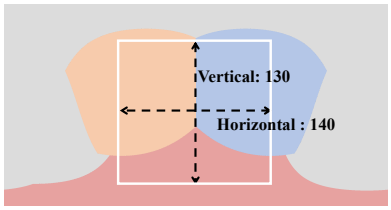


Figure 8: Partial panorama generated by a three-camera rig.

Partial panorama frame generation. We design a customized three-camera rig for TRINITY, where the cameras are placed with a fixed offset to the player's FoV to ensure coverage beyond the partial panorama boundaries as illustrated in Fig. 8. In Fig. 8, the orange, blue, and red regions represent the coverage areas of the three cameras, while the white wireframe indicates the coverage area of the partial panorama. Once generated, the partial panorama is processed using VAM (Visible Area plus Margin) [42], which rotates, re-projects, and scales the panorama into an image frame of a predefined resolution. VAM provides flexibility for varying angular coverage, compatibility with encoding & decoding, and efficiency by dynamically reprojecting as the viewing direction changes. We then apply Axis-Aligned Distortion Transmission (AADT) [34], a fixed-gaze-point compression method that preserves resolution at the visual center while downsampling the periphery to reduce bandwidth. For the captured textures, we use a customized shader to generate the partial panorama, which involves a series of matrix calculations and projection operations. The frame generation in the shader is invoked by a C# script and then attached to the *PlayerCamera* in the FPSSample scene, executing during each rendering cycle.

Encoding and transmission. After texture generation, we convert it to YUV420 format, embed the motion index into the frame, and perform hardware encoding using Nvidia's H.265 encoder [38]. In real-world deployments, we select TCP to ensure reliability under varying network conditions, as it experiences lower packet loss and fewer game stalls. We implement the streaming pipeline, encapsulate relevant APIs into dynamically linked libraries, and integrate them into Unity.

5 Evaluation

5.1 Experimental Setup

Testbed. To evaluate the impact of latency on cloud VR gaming systems and make the experiment reproducible. We setup a controlled evaluation testbed which is shown in Fig. 9.

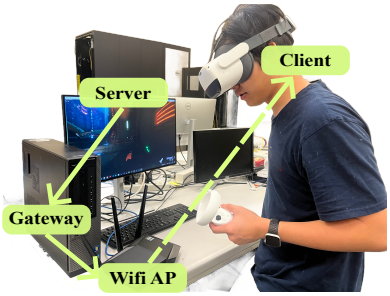


Figure 9: Experiment testbed.

The server operates on a laptop with an Nvidia RTX 4080 graphics card, an Intel i7 CPU, and 32 GB of memory. The client operates on a PICO-3 headset [39] with an 8-core CPU which is connected to a Wi-Fi 802.11ax AP. To ensure reproducibility, we include a Linux machine as a gateway between the server and WiFi AP to emulate additional network delays, simulating the effects of a remote server. The goal of the whole experiment is to demonstrate that Trinity can mitigate large MTP latency in a cloud setup under strong and stable network conditions. Therefore, we omit experiments conducted in varying network environments.

Parameters setup. To determine the parameters in equation 1, we evaluated the Trinity prototype under network latencies of 50, 70, 100, 130, 160 ms. For each latency level, we manually adjusted the delay for hand motion and rated the shooting experience on a 1–5 scale (1 = poor, 5 = excellent). The MOS results led to the following observations: (i) when body motion latency exceeded 160 ms, adding additional latency to hand motion no longer preserved QoE, as it surpassed participants' tolerance; thus, we set $MTP_{unsat} = 160$. (ii) When body motion latency (round-trip) was below 70 ms, no noticeable degradation in gaming experience was found. Therefore, we set $MTP_{sat} = 70$. (iii) We set $\alpha = 1/4$ and $\beta = 3/4$ to keep the latency gap between hand motion and body motions constant ~ 30 ms, as suggested by Fig. 6a. These parameters can be further fine-tuned or adapted to optimize the gaming experience. The partial panorama size is determined by the system's MTP latency: our measurements show a round-trip MTP of about 130 ms, and according to [42], a $140° \times 130°$ view covers over 99% of viewers' head movements under this threshold.

Baseline. We use local and remote streaming setups via ALVR [11] as two baseline schemes described in §2.1. Specifically, all rendering tasks are offloaded to the server and the result frames are streamed wirelessly [19]. For the local setup, the server and VR headset are co-located within a WLAN. For remote setup, we use our collected real-world network traffic data to emulate the network latency. The average RTT between the client and cloud server is around 40ms. We also introduce additional latency to Trinity for fair comparison. The default camera FoV and resolution are set as $100° \times 100°$ and 2800x2600. Both ALVR and Trinity run the open-source VR game FPSSample [16] and the only difference is that the shooting gun asset is rendered directly on the VR headset for Trinity while all original game logic and effect remain to be the same.

User study setup. We conduct formal user studies with 31 participants. The self-reported age range is $20 - 65$ (13 females, 18 males). They made a self-assessment regarding their experience with the FPS game and VR. Out of 31 participants, 20 people have never played VR before, 10 have played VR games for 1-3 times per week, and one person has the great VR experience and played the VR over 4 times per week. Before engagement, all participants were provided an overview of the study and consented to participate per institutional ethics and privacy policies. The user study contains several steps. First, participants were asked to familiarize themselves with the operation of VR device and the FPS game using a reference setup where no additional latency is added. After a 5 minute warm-up session, we let each participant play all three setups (local, remote, and Trinity) for 5 minutes each. To mitigate order effects, we randomized the presentation order across participants. During the session, participants were unaware of which scheme they were using and free to explore the environment (i.e., perform head and body motions) and shoot objects (hand motion). After each session, participants took a break and provided MOS scores based on their experiences from $1 - 5$: 1 indicates the worst experience while 5 indicates the best experience. Following the scoring, they had the opportunity to leave open comments.

5.2 Subjective Evaluation

Metrics. The FPSSample prototype is a non-competitive game and does not support task-performance tracking such as hit accuracy. We mainly focus on five subjective metrics that reflect the overall gaming experience [4].

- *Visual Quality* reflects resolution, texture quality, lighting, and graphical fidelity of the environment.
- *Responsiveness* measures how quickly and accurately the game reacts to player inputs such as changing the direction of view, interacting with objects or shooting, and moving through the virtual environment.
- *Artifact Effect* measures the presence and impact of visual distortions or artifacts.
- *Sickness* is a common issue in VR gaming, which includes symptoms like dizziness and nausea.
- *Overall Experience* is holistic measure of the player's overall satisfaction with the game, taking into account the above metrics.

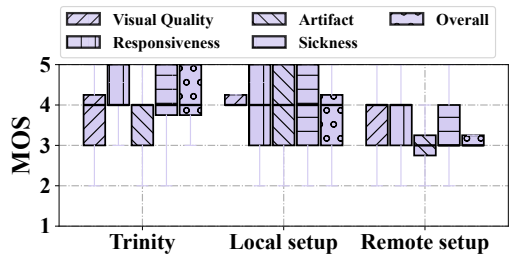


Figure 10: Subjective evaluation results: TRINITY has a comparative performance to the local streaming setup.

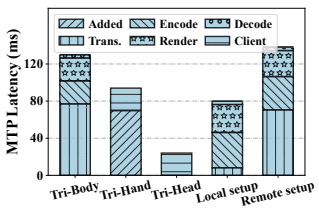


Figure 11: MTP breakdown: transmission and processing delays primarily contribute to MTP.

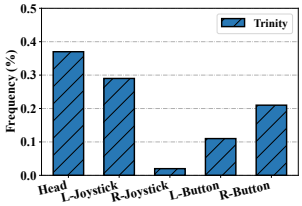


Figure 12: Motion statistics: head motion is the most frequently performed action in TRINITY.

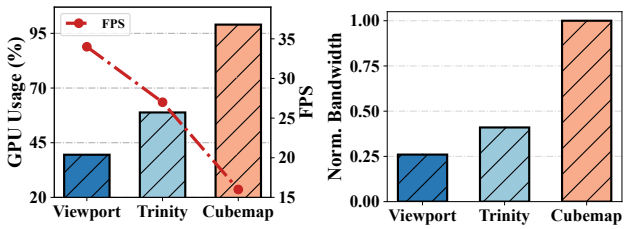
QoE results. The main subjective evaluation results are summarized in Fig. 10. We have several observations. First, compared to remote setup, TRINITY shows its great performance in all metrics on average. The gain in responsiveness is mainly due to the proposed MTP adaptation for hand motion which makes shooting the objects and moving around becomes smoother even in large round-trip network delay. The players provide the feedback that it is natural to play around using TRINITY. In addition, we notice that the sickness under remote setup is high which indicates that the large MTP can also introduce cybersickness as in [30]. The artifact score in remote setup is low since new FoV frames fail to be delivered and displayed on the VR headset in time. In contrast, players mention that they seldom see the artifact effects in TRINITY. Second, TRINITY achieves its goal by delivering performance comparable to a local streaming setup across all metrics on average. To our surprise, TRINITY even outperforms the local streaming setup in responsiveness, sickness, and overall experience. The main reason is that even at the local streaming setup, the round-trip network delay can be up to 80ms as shown in Fig. 11 due to the processing latency at the server side and propagation delay. Therefore, the participants can still feel the lag. Instead, TRINITY successfully mitigates the impact of such delay. Compared to the local setup, TRINITY has a worse artifact score because of slow updates on body motion. In addition, there is degradation in the visual quality of TRINITY compared to the baseline schemes. However, this has little impact on the overall experience as participants point out that "Responsiveness is the most noticeable difference while quality differences are minor." In summary, the results demonstrate TRINITY's ability to deliver a comparable game experience to the local setup.

5.3 Deep Dive

MTP latency. We recorded the MTP and its breakdown for each type of motion in TRINITY and the two baseline schemes, as shown in Fig. 11. Several observations can be made. First, in TRINITY, the MTP for body motion reaches approximately 130ms, while the MTP for hand motion is around 90ms. Despite these relatively high latencies, user study results (Fig. 10) indicate that the overall gaming experience and responsiveness of TRINITY are better than those in remote setup and comparable to or even better than local setup. This highlights the importance of reducing the MTP latency gap. The MTP latency for head motion is consistently kept within 25ms due to the partial panorama streaming technique. Second, we find that encoding (38ms), rendering (30ms), and transmission (70ms) are the primary contributors to the round-trip delays. The larger encoding and rendering latencies stem from the limited computational capacity of the server, though this can be easily fixed by using more powerful GPUs. Consequently, even in the local streaming setup, the round-trip network delay can reach up to 80ms.

Motion statistics. Fig. 12 illustrates the operation frequency of each motion type in TRINITY. Head motion is the most frequently performed action, accounting for 37%, followed by body motions (move left joystick) at 29%. Hand motions occur least frequently. Based on our observations from the user study, participants without VR experiences prefer to explore the environment by moving their heads, which is the easiest and most natural action for them compared to complex and unfamiliar actions like button triggering. These statistics also provide empirical evidence to prioritize the reduction of MTP latency for head motion.

Resource consumption. In this part, we highlight TRINITY's advantage in reducing resource consumption compared to existing solutions. Specifically, we compare three schemes, i.e., TRINITY, *Viewport*, and *Cubemap*. *Cubemap* [45] is a representative and practical latency-masking approach adopted by the existing system (as discussed in §2.2). *Viewport* refers to another class of existing VR streaming approaches (e.g., ALVR) that stream only the viewport. Because the original FPS game scene is highly complex and makes it difficult to isolate individual factors, we use a simpler Unity scene (Viking Village [46]) for this experiment. We adopt the constant quantization parameter approach with *QP* value being 26 and H.265 encoder. Fig. 13 illustrates GPU utilization and bandwidth consumption. TRINITY achieves reductions of approximately 40% in GPU usage and 60% in bandwidth compared to the *Cubemap* method by only generating the



(a) Computation Overhead. **(b) Bandwidth Usage.**

Figure 13: Comparisons of resource consumptions for different VR gaming systems.

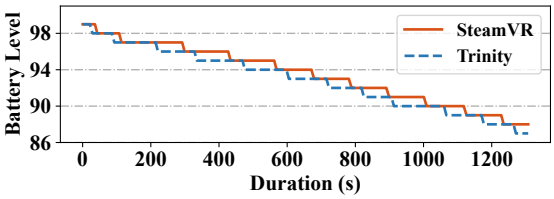


Figure 14: Battery consumption for VR headset.

partial panorama, while the *Cubemap* method must render and transmit all six faces of the cube. Compared to *Viewport*, Trinity exhibits slightly higher bandwidth and GPU utilization (resulting in lower server FPS), due to the generation and transmission of additional data. Nonetheless, such computation and transmission overload is negligible given the improvements in responsiveness and sickness. We also recorded the battery level of the VR headset when running Trinity and *Viewport* and found it only drops about 10% for 20 minutes for both schemes as illustrated in Fig. 14, indicating that local rendering is not a system bottleneck, enabling Trinity to support hours-long playback. The battery of the headset drops a little bit faster when running Trinity. The main reason is that VR headset has to perform a lightweight rendering task for the foreground objects.

6 Discussion

3DoF *vs.* 6DoF. In Trinity, we adopt partial panoramas to mitigate latency for 3-DOF head motion but do not employ image-based rendering techniques for 6-DOF motion. This decision is motivated by two factors: 1) Motions that change the viewpoint position are typically associated with body movement, which can tolerate higher latency. Reducing this latency could introduce inconsistencies between different interaction types (e.g., movement vs. firing) and does not necessarily improve overall QoE. 2) DIBR techniques face inherent challenges such as rendering artifacts (e.g., occlusion holes, shadows) and require significant additional computing and networking resources, that are already heavily consumed by generating partial panoramas.

Generalize to other applications. While FPS games naturally involve straightforward first-person interactions, such as shooting or picking up items, our framework is not limited to this genre. It can be readily applied to other games (e.g., car racing), provided they feature contrasting and easily distinguishable foreground elements against the background scene.

Limitations. First, while our results demonstrate the effectiveness of Trinity in an FPS game, the current design will encounter new challenges as VR applications become more complex and strive for greater realism. For instance, the latest VR headsets can capture not only head rotation but also subtle head translations in 3D space. Supporting applications that rely on these fine-grained movements will require reconsidering strategies for handling 6-DOF motion. The interaction of Trinity can be further improved by replacing joystick-based positional control with real physical movement. Second, the hand-motion adaptation model could be further improved by leveraging machine-learning-based approaches, which we leave for future investigation.

Furthermore, Trinity is not easily integrated with off-the-shelf VR games. Integration often demands source-code-level customization to extract hand-motion objects for local rendering on the headset. In many cases, this may require substantial redesign, ensuring that hand-motion objects are rendered separately from background scenes and remain within the headset's rendering capabilities.

7 Conclusion

This paper introduced Trinity, a practical solution for streaming FPS games from cloud servers to VR headsets, delivering performance comparable to—or even surpassing—local streaming setups. The key innovation of Trinity lies in exploiting players' varying latency sensitivities across different types of interactions. Beyond FPS gaming, the idea of inserting delayed locally rendered objects or animations to bridge latency gaps can be generalized to other game genres and interactive applications. Looking ahead, we plan to extend Trinity to support a broader range of VR experiences and explore its applicability to emerging scenarios where real-time responsiveness is critical.

8 Acknowledgement

This research was funded by NSF CNS 21-06592, CCF 22-17144, CNS 19-00875. The opinions, findings, and conclusions or recommendations expressed in this paper are those of the authors and do not necessarily reflect the views of the NSF.

References

[1] A2D Software. 2024. Subside. https://store.steampowered.com/app/2550040/Subside/. (2024).
[2] Divergent Realities, LLC. 2023. Cards & Tankards. https://store.steampowered.com/app/1506850/Cards__Tankards/. (2023).

[3] Gaijin Entertainment. 2013. War Thunder. https://store.steampowered.com/app/236390/War_Thunder/. (2013).

[4] Lingzhi Zhao. 2025. VR questionnaire for subjective evaluation. https://docs.google.com/forms/d/e/1FAIpQLScm_VqdR8w4NqWQCKuwuBnWTb7eLJ9VzCgNGK3Fe4mtVD71gQ/viewform?usp=sf_link. (2025).

[5] Shu Mei Deng, Sahrish Javed, Julie Tan with Natalie Weng. 2005. Fingertip Reaction Time. https://store.steampowered.com/app/236390/War_Thunder/. (2005).

[6] Steam. 2015. Steam Link. https://store.steampowered.com/app/353380/Steam_Link/. (2015).

[7] Omid Abari, Dinesh Bharadia, Austin Duffield, and Dina Katabi. 2016. Cutting the Cord in Virtual Reality. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets '16)*. Association for Computing Machinery, New York, NY, USA, 162–168. https://doi.org/10.1145/3005745.3005770

[8] Omid Abari, Dinesh Bharadia, Austin Duffield, and Dina Katabi. 2017. Enabling High-Quality Untethered Virtual Reality. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, Boston, MA, 531–544. https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/abari

[9] Bernard D Adelstein, Thomas G Lee, and Stephen R Ellis. 2003. Head tracking latency in virtual environments: psychophysics and a model. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, Vol. 47. SAGE Publications Sage CA: Los Angeles, CA, 2083–2087.

[10] Ahmad Alhilal, Ze Wu, Yuk Hang Tsui, and Pan Hui. 2024. FovOptix: Human Vision-Compatible Video Encoding and Adaptive Streaming in VR Cloud Gaming. In *Proceedings of the 15th ACM Multimedia Systems Conference (MMSys '24)*. Association for Computing Machinery, New York, NY, USA, 67–77. https://doi.org/10.1145/3625468.3647612

[11] ALVR. Jul-2024. ALVR - Air Light VR. https://github.com/alvr-org/ALVR. (Jul-2024).

[12] Wyeth Bair, James R. Cavanaugh, Matthew A. Smith, and J. Anthony Movshon. 2002. The timing of response onset and offset in macaque visual neurons. *The Journal of Neuroscience* 22, 8 (Apr 15 2002), 3189–3205. https://doi.org/10.1523/JNEUROSCI.22-08-03189.2002

[13] Kevin Boos, David Chu, and Eduardo Cuervo. 2016. FlashBack: Immersive Virtual Reality on Mobile Devices via Rendering Memoization. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '16)*. Association for Computing Machinery, New York, NY, USA, 291–304. https://doi.org/10.1145/2906388.2906418

[14] Brian Hart. 2014. What VR Could, Should, and Almost Certainly Will Be Within Two Years. https://medium.com/silicon-valley-tech-life/what-vr-could-should-and-almost-certainly-will-be-within-two-years-fd95f89f0332. (2014).

[15] Enguang Fan, Emerson Sie, Federico Cifuentes-Urtubey, and Deepak Vasisht. 2025. Poster: Scalable Indoor Localization with Non-Cooperative Wi-Fi Ranging. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking (ACM MOBICOM '25)*. Association for Computing Machinery, New York, NY, USA, 1341–1343. https://doi.org/10.1145/3680207.3765680

[16] FPS Sample. 2024. FPS Sample. https://github.com/Unity-Technologies/FPSSample. (2024).

[17] Gabriel Zamora. 2024. The Best VR Games for 2024. https://www.pcmag.com/picks/the-best-vr-games. (2024).

[18] GEAR VR. 2017. Samsung GearVR. https://en.wikipedia.org/wiki/Samsung_Gear_VR. (2017).

[19] Jin Heo, Ketan Bhardwaj, and Ada Gavrilovska. 2023. FleXR: A System Enabling Flexibly Distributed Extended Reality. In *Proceedings of the 14th ACM Multimedia Systems Conference (MMSys '23)*. Association for Computing Machinery, New York, NY, USA, 1–13. https://doi.org/10.1145/3587819.3590966

[20] HTC. Dec-2023. Compare VIVE Systems. . https://www.vive.com/us/product/comparison/. (Dec-2023).

[21] Gazi Karam Illahi, Matti Siekkinen, Teemu Kämäräinen, and Antti Ylä-Jääski. 2021. Foveated streaming of real-time graphics. In *Proceedings of the 12th ACM Multimedia Systems Conference (MMSys '21)*. Association for Computing Machinery, New York, NY, USA, 214–226. https://doi.org/10.1145/3458305.3463383

[22] Jason J Jerald. 2009. *Scene-motion-and latency-perception thresholds for head-mounted displays*. Ph.D. Dissertation. The University of North Carolina at Chapel Hill.

[23] Jordan Phang. 2023. The State of the FPS Market. https://naavik.co/digest/state-of-fps-market/. (2023).

[24] Teemu Kämäräinen, Matti Siekkinen, Jukka Eerikäinen, and Antti Ylä-Jääski. 2018. CloudVR: Cloud Accelerated Interactive Mobile Virtual Reality. In *Proceedings of the 26th ACM International Conference on Multimedia (MM '18)*. Association for Computing Machinery, New York, NY, USA, 1181–1189. https://doi.org/10.1145/3240508.3240620

[25] Teemu Kämäräinen, Matti Siekkinen, Antti Ylä-Jääski, Wenxiao Zhang, and Pan Hui. 2017. A Measurement Study on Achieving Imperceptible Latency in Mobile Cloud Gaming. In *Proceedings of the 8th ACM on Multimedia Systems Conference (MMSys'17)*. Association for Computing Machinery, New York, NY, USA, 88–99. https://doi.org/10.1145/3083187.3083191

[26] Isaac L. Kurtzer. 2015. Long-latency reflexes account for limb biomechanics through several supraspinal pathways. *Frontiers in Integrative Neuroscience* 8 (jan 29 2015), 99. https://doi.org/10.3389/fnint.2014.00099

[27] Zeqi Lai, Y. Charlie Hu, Yong Cui, Linhui Sun, and Ningwei Dai. 2017. Furion: Engineering High-Quality Immersive Virtual Reality on Today's Mobile Devices. In *Proceedings of the 23rd Annual International Conference on Mobile Computing and Networking (MobiCom '17)*. Association for Computing Machinery, New York, NY, USA, 409–421.

[28] Kyungmin Lee, David Chu, Eduardo Cuervo, Johannes Kopf, Yury Degtyarev, Sergey Grizan, Alec Wolman, and Jason Flinn. 2015. Outatime: Using Speculation to Enable Low-Latency Continuous Interaction for Mobile Cloud Gaming. In *Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '15)*. Association for Computing Machinery, New York, NY, USA, 151–165. https://doi.org/10.1145/2742647.2742656

[29] Luyang Liu, Ruiguang Zhong, Wuyang Zhang, Yunxin Liu, Jiansong Zhang, Lintao Zhang, and Marco Gruteser. 2018. Cutting the Cord: Designing a High-quality Untethered VR System with Low Latency Remote Rendering. In *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '18)*. Association for Computing Machinery, New York, NY, USA, 68–80. https://doi.org/10.1145/3210240.3210313

[30] Shengmei Liu and Mark Claypool. 2023. The Impact of Latency on Target Selection in First-Person Shooter Games. In *Proceedings of the 14th ACM Multimedia Systems Conference (MMSys '23)*. Association for Computing Machinery, New York, NY, USA, 51–61. https://doi.org/10.1145/3587819.3590977

[31] Shengmei Liu, Xiaokun Xu, and Mark Claypool. 2022. A Survey and Taxonomy of Latency Compensation Techniques for Network Computer Games. *ACM Comput. Surv.* 54, 11s, Article 243 (sep 2022), 34 pages. https://doi.org/10.1145/3519023

[32] Yixiang Mao, Liyang Sun, Yong Liu, and Yao Wang. 2020. Low-latency fov-adaptive coding and streaming for interactive 360 video streaming. In *Proceedings of the 28th ACM International Conference on Multimedia*. 3696–3704.

[33] Jiayi Meng, Sibendu Paul, and Y. Charlie Hu. 2020. Coterie: Exploiting Frame Similarity to Enable High-Quality Multiplayer VR on Commodity Mobile Devices. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*. Association for Computing Machinery, New York, NY, USA, 923–937. https://doi.org/10.1145/3373376.3378516

[34] Meta. 2019. How does Oculus Link Work? The Architecture, Pipeline and AADT Explained. https://developers.meta.com/horizon/blog/how-does-oculus-link-work-the-architecture-pipeline-and-aadt-explained/. (2019).

[35] Meta. 2021. Introducing Application SpaceWarp. https://developers.meta.com/horizon/blog/introducing-application-spacewarp/. (2021).

[36] Meta. 2023. Expand your world with Meta Quest. https://www.meta.com/quest/. (2023).

[37] Meta. 2023. This is Meta Quest Pro . https://www.meta.com/quest/quest-pro/. (2023).

[38] NVDIA. 2024. Explore the Future of Visual Computing for Professionals. https://www.nvidia.com/en-us/n. (2024).

[39] PICO. 2023. This is Pico Neo3 . https://www.picoxr.com/global/products/neo3-link. (2023).

[40] Arunkumar Ravichandran, Ish Kumar Jain, Rana Hegazy, Teng Wei, and Dinesh Bharadia. 2018. Facilitating Low Latency and Reliable VR over Heterogeneous Wireless Networks. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom '18)*. Association for Computing Machinery, New York, NY, USA, 723–725. https://doi.org/10.1145/3241539.3267781

[41] Shu Shi, Varun Gupta, Michael Hwang, and Rittwik Jana. 2019. Mobile VR on edge cloud: a latency-driven design. In *Proceedings of the 10th ACM Multimedia Systems Conference (MMSys '19)*. Association for Computing Machinery, New York, NY, USA, 222–231. https://doi.org/10.1145/3304109.3306217

[42] Shu Shi, Varun Gupta, and Rittwik Jana. 2019. Freedom: Fast Recovery Enhanced VR Delivery Over Mobile Networks. In *Proceedings of the 17th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '19)*. Association for Computing Machinery, New York, NY, USA, 130–141. https://doi.org/10.1145/3307334.3326087

[43] SteamVR. 2024. SteamVR. https://store.steampowered.com/app/250820/SteamVR/. (2024).

[44] Zhaowei Tan, Yuanjie Li, Qianru Li, Zhehui Zhang, Zhehan Li, and Songwu Lu. 2018. Supporting Mobile VR in LTE Networks: How Close Are We? *Proc. ACM Meas. Anal. Comput. Syst.* 2, 1, Article 8 (2018), 31 pages. https://doi.org/10.1145/3179411

[45] Unity. 2017. Cubemap. https://docs.unity3d.com/550/Documentation/Manual/class-Cubemap.html. (2017).

[46] Unity. 2024. Viking Village URP. https://assetstore.unity.com/packages/essentials/tutorial-projects/viking-village-urp-29140?srsltid=AfmBOoquwGdV10QP1tfuPVaN-N3_GaNMbFhybrkkcnIPoJGeQmOLPY2x. (2024).

[47] Sam Van Damme, Maria Torres Vega, Joris Heyse, Femke De Backere, and Filip De Turck. 2020. A low-complexity psychometric curve-fitting approach for the objective quality assessment of streamed game videos. *Signal processing: image communication* 88 (2020), 115954.

[48] Verified Market Reports. 2025. VR Shooting Games Market Insights. https://www.verifiedmarketreports.com/product/vr-shooting-games-market/?utm_source=chatgpt.com/. (2025).

[49] Shuoqian Wang, Xiaoyang Zhang, Mengbai Xiao, Kenneth Chiu, and Yao Liu. 2020. Sphericrtc: A system for content-adaptive real-time 360-degree video communication. In *Proceedings of the 28th ACM International Conference on Multimedia*. 3595–3603.

[50] Jiangkai Wu, Yu Guan, Qi Mao, Yong Cui, Zongming Guo, and Xinggong Zhang. 2023. ZGaming: Zero-Latency 3D Cloud Gaming by Image Prediction. In *Proceedings of the ACM SIGCOMM 2023 Conference (ACM SIGCOMM '23)*. Association for Computing Machinery, New York, NY, USA, 710–723. https://doi.org/10.1145/3603269.3604819

[51] Lingzhi Zhao, Ying Cui, Zhi Liu, Yunfei Zhang, and Sheng Yang. 2021. Adaptive Streaming of 360 Videos With Perfect, Imperfect, and Unknown FoV Viewing Probabilities in Wireless Networks. *IEEE Transactions on Image Processing* 30 (2021), 7744–7759. https://doi.org/10.1109/TIP.2021.3099741